CHAPTER 10

Internal Rate of Return & Numerical Analysis

Polynomial Formulations, The Abel-Ruffini Theorem, Newton-Raphson Schemes, and Descartes' Rule of Signs

---

10.1 The IRR as a Polynomial Root-Finding Problem

The Internal Rate of Return (IRR) is the effective annualized yield rate that equates the Net Present Value of a multi-period investment stream to zero. Mathematically, it represents the real root r^* of the non-linear equation:

Definition 10.1 (The Internal Rate of Return Equation)

Given cash flows $\{C_0, C_1, ..., C_T\}$, the Internal Rate of Return r^* is the solution satisfying:

$$f(r) = \text{NPV}(r) = \sum_{t=0}^{T} \left[C_t / (1 + r)^t \right] = 0$$

Substituting $x = 1 + r$ transforms $f(r) = 0$ into a degree-T algebraic polynomial:
$P(x) = C_0 x^T + C_1 x^{T-1} + ... + C_T = 0.$

10.2 Why Numerical Methods Are Mathematically Obligatory

The Abel-Ruffini Impossibility Theorem: In pure algebra, polynomials of degree $T \le 4$ possess closed-form solutions via radicals (quadratic, cubic Cardano, and quartic Ferrari formulas). However, Galois theory establishes that the symmetric permutation group S_n is non-solvable for $n \ge 5$. Consequently, for any financial project exceeding 4 years ($T \ge 5$), **no general algebraic radical formula exists** to solve for IRR! Numerical analysis is not an engineering shortcut—it is an absolute mathematical necessity.

Theorem 10.1 (Newton-Raphson Iterative Formulation & Analytical Derivative)

Newton-Raphson generates a sequence $\{r_k\}$ converging quadratically to root r^*:
$$\boldsymbol{r_{k+1} = r_k - \left[f(r_k) / f'(r_k) \right]}$$
where the exact first derivative is derived analytically without finite-difference errors:
$$\boldsymbol{f'(r) = -\sum_{t=1}^{T} \left[t \cdot C_t / (1 + r)^{t+1} \right]}$$

Quadratic Convergence: Error satisfies $|e_{k+1}| \approx |f''(r^*) / (2f'(r^*))| \cdot |e_k|^2$, doubling the number of valid significant digits at every single iteration.

10.3 Step-by-Step Worked Examples

Below are two formal case studies solved by hand: full Newton-Raphson manual iteration trace, and resolving multiple non-unique IRRs under Descartes' Rule of Signs.

Example 10.1: Full Manual Newton-Raphson Iteration Trace

Problem Statement: An investment requires outlay C_0 = –**INR 100,000** and produces three annual inflows: C_1 = **INR 35,000**, C_2 = **INR 45,000**, and C_3 = **INR 45,000**. Using starting initial guess r_0 = **0.100000 (10.00%)**, trace the Newton-Raphson sequence by hand until convergence is established to four decimal places.

Step-by-Step Analytical Solution:

Iteration 0 (Evaluate at $r_0 = 0.10$):

- $f(0.10) = -100{,}000 + 35{,}000/(1.1) + 45{,}000/(1.1)^2 + 45{,}000/(1.1)^3$
  $= -100{,}000 + 31{,}818.18 + 37{,}190.08 + 33{,}809.17 =$ **+INR 2,817.43**.
- $f'(0.10) = -\left[\, 1\times35{,}000/(1.1)^2 + 2\times45{,}000/(1.1)^3 + 3\times45{,}000/(1.1)^4 \,\right]$
  $= -\left[\, 28{,}925.62 + 67{,}618.33 + 92{,}206.82 \,\right] =$ **–188,750.77**.
- Update: $r_1 = r_0 - [\, f(r_0) / f'(r_0) \,] = 0.10 - [\, 2{,}817.43 / (-188{,}750.77) \,] = 0.10 + 0.014927 =$ **0.114927**.

Iteration 1 (Evaluate at $r_1 = 0.114927$):

- $1 + r_1 = 1.114927$.
- $f(r_1) = -100{,}000 + 35{,}000/(1.114927) + 45{,}000/(1.114927)^2 + 45{,}000/(1.114927)^3$
  $= -100{,}000 + 31{,}392.20 + 36{,}200.74 + 32{,}469.17 =$ **+INR 62.11**.
- $f'(r_1) = -181{,}940.35$.
- Update: $r_2 = 0.114927 - [\, 62.11 / (-181{,}940.35) \,] = 0.114927 + 0.000341 =$ **0.115268**.

Iteration 2 (Evaluate at $r_2 = 0.115268$):

- $f(r_2) =$ **+INR 0.03** ≈ 0.00.
- **Converged IRR:** r^* = **0.11527 &implies; 11.53% per annum** (converged in 2 steps!).

Example 10.2: Descartes' Rule of Signs & The Multiple IRR Paradox

Problem Statement: A natural resource strip-mining venture has an initial setup cost of C_0 = –**INR 100,000**, generates operational cash flow of C_1 = **+INR 230,000** in Year 1, and incurs environmental remediation decommissioning costs of C_2 = –**INR 132,000** in Year 2. Apply Descartes' Rule of Signs, determine all real IRRs, and discuss the analytical implications.

Step-by-Step Analytical Solution:

Step 1 (Descartes' Rule of Signs Analysis):

Examine the sequence of cash flow signs: [–, +, –].

- Sign transitions: From – to + (1st change), and + to – (2nd change).
- Total sign changes = 2.
- By Descartes' Rule of Signs, the polynomial equation has either **2 or 0 positive real roots**.

Step 2 (Exact Algebraic Root Determination):

Set NPV equal to zero: $-100{,}000 + 230{,}000/(1+r) - 132{,}000/(1+r)^2 = 0$.

Let $x = 1 / (1 + r)$. The equation becomes quadratic:

$-100{,}000 + 230{,}000x - 132{,}000x^2 = 0$.

Dividing by –2,000: $66x^2 - 115x + 50 = 0$.

Factoring: $(11x - 10)(6x - 5) = 0$.

- Root 1: $x_1 = 10 / 11 \Rightarrow 1 + r_1 = 11 / 10 = 1.1000 \Rightarrow$ **$r_1^* = 10.00\%$**.
- Root 2: $x_2 = 5 / 6 \Rightarrow 1 + r_2 = 6 / 5 = 1.2000 \Rightarrow$ **$r_2^* = 20.00\%$**.

Step 3 (Financial Decision Paradox):

- Both 10.0% and 20.0% yield exact NPV = 0! For $r \in$ (10%, 20%), NPV > 0, while outside, NPV < 0.
- *Takeaway:* Multiple IRRs destroy the credibility of IRR for non-conventional cash flows. A decision support system must detect sign variations and advise fallback to Net Present Value.

10.4 Algorithmic Implementation in Python

Below is the complete implementation from models/capital_budgeting.py, featuring Newton-Raphson iteration with exact analytical derivatives and Bisection bracket fallback:

Listing: models/capital_budgeting.py — Hybrid Numerical IRR Engine

```
from typing import List, Optional

def irr_newton_raphson(
    cash_flows: List[float], initial_guess: float = 0.10, tolerance: float = 1e-7, max_iter: int = 100
) -> Optional[float]:
    """
    Solves for IRR using Newton-Raphson with exact analytical derivative: f'(r) = -sum(t*C_t/(1+r)**(t+1)).
    """
    r = initial_guess
    for _ in range(max_iter):
        f_val = sum(cf / ((1.0 + r) ** t) for t, cf in enumerate(cash_flows))
        f_prime = sum(-t * cf / ((1.0 + r) ** (t + 1)) for t, cf in enumerate(cash_flows) if t > 0)
        if abs(f_prime) < 1e-12:
            return None  # Derivative near zero: switch to Bisection
        r_next = r - (f_val / f_prime)
        if abs(r_next - r) < tolerance:
            return float(r_next)
        r = r_next
    return None


def irr_bisection(
    cash_flows: List[float], a: float = -0.50, b: float = 2.0, tolerance: float = 1e-7, max_iter: int = 100
) -> Optional[float]:
    """Guaranteed convergence fallback bracketed solver: c = (a + b) / 2."""
    def npv(rate: float) -> float:
        return sum(cf / ((1.0 + rate) ** t) for t, cf in enumerate(cash_flows))
    fa, fb = npv(a), npv(b)
    if fa * fb > 0:
        return None
    for _ in range(max_iter):
        c = 0.5 * (a + b)
        fc = npv(c)
        if abs(fc) < tolerance or 0.5 * (b - a) < tolerance:
            return float(c)
        if fa * fc < 0:
            b, fb = c, fc
        else:
            a, fa = c, fc
    return float(0.5 * (a + b))
```

10.5 Mathematical Properties Summary

Mathematical Property	Specification & Formula	Analytical Significance
Root Definition	NPV(r^*) = 0	Internal yield rate balancing discounted outflows and inflows
Abel-Ruffini Bound	Non-solvable for $T \geq 5$	No closed-form algebraic radical formulas exist; mandates numerical schemes
Newton-Raphson Order	Order 2 (Quadratic)	Doubles significant decimal precision per iterative step
Bisection Convergence	Linear ($\lvert e_k \rvert \leq 2^{-k}(b-a)$)	Unconditional convergence guaranteed whenever bracket sign flips
Descartes' Bound	Roots ≤ Sign Changes	Detects potential multiple IRR ambiguities in non-conventional cash flows